FXML

Dany Siriphol – CODA 1 ère année - 2025

Introduction à FXML

FXML est un langage basé sur XML qui permet de décrire une interface graphique JavaFX

```
<VBox minWidth="200"  spacing="10" alignment="CENTER"
    style="-fx-padding: 10px; -fx-background-color: #FFFFFF;">
    <Label text="Creer une tâche" style="-fx-font-size: 16px;"/>
    <TextField fx:id="taskName"
        promptText="Entrez une tâche..."
        maxWidth="200"/>
    <Button text="+ Ajouter" onAction="#addTask"/>
</VBox>
```

Exemple de FXML

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.*?>
<?import java.util.*?>
<?import javafx.scene.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<AnchorPane id="AnchorPane"
    prefHeight="400.0"
    prefWidth="600.0"
    xmlns:fx="http://javafx.com/fxml">
</AnchorPane>
```

Chargement d’un FXML

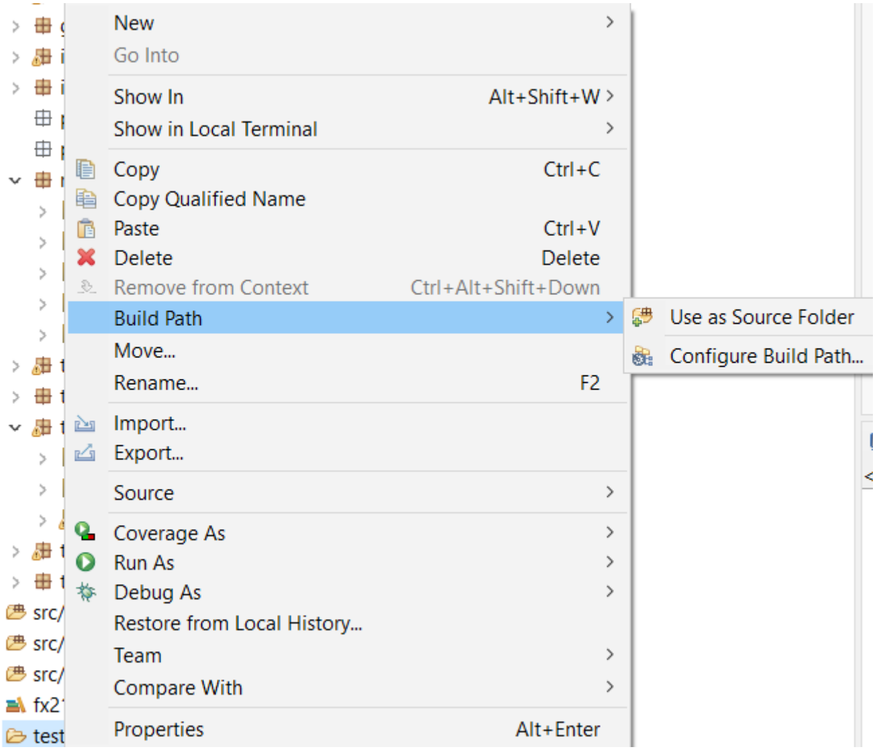
```
@Override
public void start(Stage primaryStage) {
    try {
        URL url = getClass().getResource("test.fxml");
        FXMLLoader fxmlLoader = new FXMLLoader(url);

        final Parent root = (Parent) fxmlLoader.load();
        final Scene scene = new Scene(root, 500, 500);
        primaryStage.setScene(scene);
    } catch (IOException e) {
        System.err.println("Erreur au chargement: " + e);
    }
    primaryStage.setTitle("Test FXML");
    primaryStage.show();
}
```

assets d’un projet Eclipse

Si votre projet utilise des assets, il est conseillé de créer un dossier dédié sur votre projet

- Créer un dossier à la racine de votre projet
- ajouter le dossier dans le build path (Eclipse)
- configurer le dossier comme un dossier source (IntelliJ)



Controller FXML

Dany Siriphol – CODA 1 ère année - 2025

Controller

Un controller est une classe Java qui gère la logique métier et les interactions avec l’interface graphique

→ Séparer le design du comportement

```
<BorderPane xmlns="http://javafx.com/javafx/8"
    xmlns:fx="http://javafx.com/fxml/1"
   fx:controller="todolist.TodoController"
   style="-fx-padding: 10px;">
```

Controller

Un controller est une classe Java qui gère la logique métier et les interactions avec l’interface graphique

```
<BorderPane xmlns="http://javafx.com/javafx/8"
    xmlns:fx="http://javafx.com/fxml/1"
   fx:controller="todolist.TodoController"
   style="-fx-padding: 10px;">
```

Structure d’un Controller

Le controller est une classe attaché à un FXML. Elle possède une méthode initialize appelé au démarrage de l’application

```
public class TodoController {


    @FXML
    public void initialize(){
    }


}
```

Controller – accéder aux elements FXML

```
<TextField fx:id="taskName" maxWidth="200"/>

<VBox fx:id="taskList" prefHeight="400"/>
```

```
public class TodoController {

    @FXML private TextField taskName;
    @FXML private VBox taskList;
```

Gestion des évènements

```
<Button text="+ Ajouter" onAction="#addTask"/>
```

```
public class TodoController {

    @FXML
    private void addTask() {
        // ...
    }
```

Style avec CSS

Dany Siriphol – CODA 1 ère année - 2025

Style avec CSS

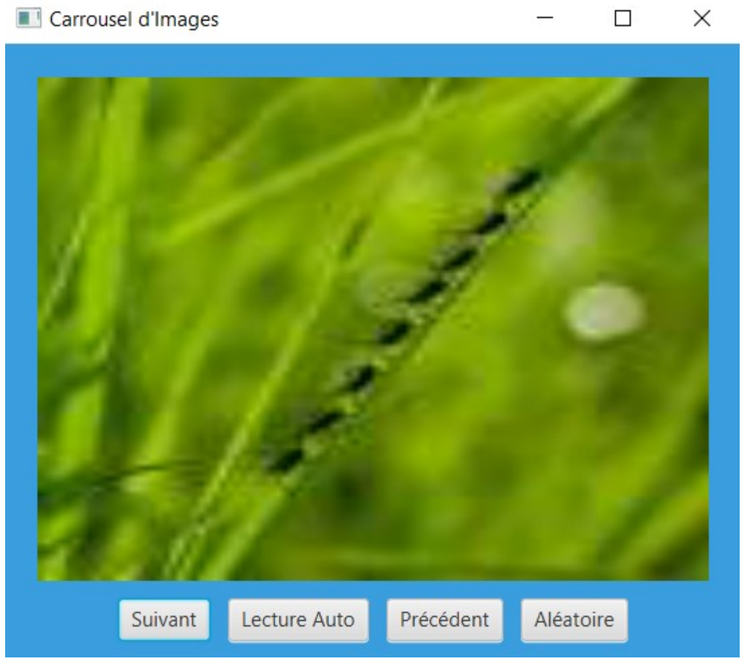
JavaFX possède la faculté d’utiliser des scripts CSS pour définir le style des éléments

```
scene.getStylesheets().add(
        getClass().getResource("/style.css").toExternalForm()
);
```

Style avec CSS

```
selector { property: value }
```

```
.root {
    -fx-background-color: #3498db; /* Bleu */
}
```



CSS inline

```
<Label text="Creer une tâche" style="-fx-font-size: 16px;"/>
```

```
button1.setStyle("-fx-background-color: darkslateblue;");
button2.setStyle("-fx-text-fill: white;");
```

Syntaxe CSS : classe

En JavaFX, pour appliquer des styles sur les éléments de base ; il faut prefixer toutes les règles par un point(.) car ils sont considéré comme des classes

```
.root {
    -fx-background-color: #3498db; /* Bleu */
}

.button {
    -fx-background-color: blue;
    -fx-text-fill: white;
}
```

Syntaxe CSS : classe

Il est possible d'ajouter des classes aux éléments et appliquer leur appliquer des règles CSS

```
<Button text="Valider" styleClass="validate"/>
```

```
Button b = new Button("Valider");

// Ajouter une classe CSS
b.getStyleClass().add("A");

// Retirer une classe CSS
b.getStyleClass().remove("B");

// Remplacer une classe CSS
b.getStyleClass().removeAll("A");
b.getStyleClass().add("B");
```

```
.validate {
    -fx-background-color: #3498db;
    -fx-text-fill: white;
    -fx-font-size: 16px;
    -fx-border-radius: 5px;
}
```

Syntaxe CSS : id

```
<Button text="Valider" fx:id="myID"/>
```

```
b.setId("myID");
```

```
#myID {
    -fx-background-color: #3498db;
    -fx-text-fill: white;
    -fx-font-size: 16px;
    -fx-border-radius: 5px;
}
```

Réference

Références CSS :

https://openjfx.io/javadoc/19/javafx.graphics/javafx/scene/doc-files/cssref.html

Créer un composant FXML

Dany Siriphol – CODA 1 ère année - 2025

Créer ses propres composants

En JavaFX, il est possible de créer ses propres composants réutilisable dans l’application

```
<VBox xmlns:fx="http://javafx.com/fxml"
fx:controller="carrousel.CarrouselController"
      spacing="10" alignment="CENTER" style="-fx-padding: 20px;">
    <MyComponent text="Hello World!"/>
```

Créer ses propres composants

Il faut tout d’abord créer un composant FXML qui possède une balise fx:root et définit le type de container utilisé par le composant

```
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<fx:root type="javafx.scene.layout.VBox"
    xmlns:fx="http://javafx.com/fxml">
    <TextField fx:id="textField"/>
    <Button text="Valider" onAction="#click"/>
</fx:root>
```

Créer ses propres composants

Il faut ensuite créer une classe qui va servir de controller. Cette classe doit étendre le container utilisé par le FXML

La classe doit charger le fichier FXML dans son constructeur et configurer la root et le controller du composant

Cette classe controller se comporte comme toutes les autres et peux accéder aux éléments du fichier FXML

```
public class MyComponent extends VBox {
    @FXML private TextField textField;

    public MyComponent() {
        FXMLLoader fxmlLoader = new FXMLLoader(
                getClass()
                .getClassLoader()
                .getResource("MyComponent.fxml")
        );
        fxmlLoader.setRoot(this);
        fxmlLoader.setController(this);

        try {
            fxmlLoader.load();
        } catch (IOException exception) {
            throw new RuntimeException(exception);
        }
    }
```

Créer ses propres composants

Le composant est maintenant utilisable dans un FXML ou directement en dans le code Java

```
FXMLLoader loader = new FXMLLoader(fxmlFile);
VBox root = loader.load();

var c = new MyComponent();
root.getChildren().add(c);
```

```
<VBox xmlns:fx="http://javafx.com/fxml"
fx:controller="carrousel.CarrouselController"
      spacing="10" alignment="CENTER" style="-fx-padding: 20px;">
    <MyComponent text="Hello World!"/>
```

Propriétés personnalisés

Il est possible de définir ses propres propriétés sur ses composants

```
<MyComponent text="Hello World!"/>
```

Si une propriété non native est utilisée, JavaFX va tenter d’appeler un setter dans la classe controller.

```
public class MyComponent extends VBox {
    @FXML private TextField textField;

    ...

    public void setText(String value) {
        textField.textProperty().set(value);
    }

    ...
}
```

Exercices